## ASSIGNMENT 1: Campus Smart-Parking System

### Overview

- **Course Outcome:** CO1 - Java fundamentals & Object-Oriented Programming.

Name: Revi Thimma Reddy kc
Reg No: 192325025

- **Total Marks:** 50
- **Duration:** 1 Week
- **Submission Requirements:**

**GitHub Repository:**

https://github.com/ReviThimmaReddy/CSA0909-JAVA containing task folders (Task01, Task02, ...), source code, execution screenshots, and progressive commit history.

b. **Document Report:** Comprehensive analysis covering Tasks 1–12, including system design, implementation code, verified test cases, and algorithmic complexity analysis.

## TASK 1: Understand the Problem and Identify the Objects

### Inputs and Outputs Analysis

The Campus Smart-Parking System processes incoming vehicles, assigns allocation spots, issues tracking tickets, and calculates billing fees upon exit.

- **Inputs:** Vehicle Registration Number (String), Vehicle Type

(TwoWheeler, Car, ElectricCar), Entry Time, Exit Time/Duration (Minutes), Pass Status (Staff/Student vs. Visitor), Charging Consumption (kWh for EVs).

- **Outputs:** Allocated Slot Details (Floor & Slot Index), Parking Ticket (Ticket Serial Number, Entry Timestamp), Fee Receipt (Bill Duration, Calculated Total Amount).

**Candidate Classes & Domain Responsibilities**

| Class Name | Domain Responsibilities | Attributes & Methods | |
|---|---|---|---|
| ParkingFacility | Manages global multi floor storage, allocation strategies, and slot release routines. | slots[][], allocate(), release() | |
| Vehicle *(Abstract)* | Provides a core identity contract for generic vehicles entering the facility. | regNo, getRegNo(), calculateFee() | |
| TwoWheeler | Implements specialized billing and attributes for motorcycles and scooters. | Overridden rate methods | |
| Car | Implements standard four-wheeler parking routines and billing rates. | Extended vehicle methods | |

| | | | |
|---|---|---|---|
| ElectricCar | Subclass of Car supporting EV auxiliary charging options. | chargingFee(), kWh tracking | |
| ParkingTicket | Encapsulates immutable transaction records created upon vehicle allocation. | ticketId, entryTime, vehicleRef | |

**Key Business Rules**

1. **Slot Exclusivity:** A single parking slot can host at most one vehicle at any given instant.
2. **Serial Identification:** Ticket identifiers must be unique, immutable, and generated via dynamic auto-incrementing counters. 3. **Pass Exemptions:** Active campus pass holders (Staff/Student) bypass standard time-based parking fees.

**Alternative Architectural Concepts & Complexity Analysis** When mapping vehicles to slots, a developer might consider **Parallel Arrays** versus **Object-Oriented Encapsulation**.

- **Parallel Arrays Approach:** Maintaining separate primitive collections (String[] regNos, int[] vehicleTypes, long[] entryTimes) requires synchronized index manipulation. Searching for a vehicle's slot takes $O(N)$ time. Modifying attributes across unlinked arrays introduces serious risks of state desynchronization.

- **Object Representation:** Grouping domain attributes into explicit object instances preserves logical state integrity. Searching through a single dimensional or multidimensional object array retains $O(N)$ traversal complexity, but yields $O(1)$ lookup capabilities when paired with a hash-based indexing structure (HashMap<String, Slot>).

**TASK 2: Choose the Data Representation**

**Primitive and Object Data Type Mapping**

- **Floor & Slot Indexes:** int for compact, non-negative integer representation.
- **Billing Amounts & kWh:** double to handle precise fractional monetary units and power consumption.
- **Vehicle Identification:** String to support alphanumeric license plate representations.
- **Temporal Tracking:** java.time.LocalTime for robust, standard time tracking, or primitive int storing elapsed duration in total minutes.

**Array Structural Selection**

A **2D Jagged Array** (String[][] slots or Slot[][] slots) is selected to represent multi-floor parking structure layouts. Real-world architecture often features variable capacity across different levels (e.g., Ground floor accommodating 50 vehicles, Basements accommodating 30). Standard rectangular arrays force equal allocation sizes, whereas jagged arrays optimize memory by instantiating exact capacity per level.// Example Jagged Array Construction

```
int[] floorCapacities = {50, 30, 20}; // Floors 0, 1, 2

String[][] parkingStructure = new String[floorCapacities.length][];

for (int i = 0; i < floorCapacities.length; i++) {

 parkingStructure[i] = new String[floorCapacities[i]]; }
```

**Casting & Precision Loss Risks**

Using integer division for duration-to-hour calculations (such as minutes / 60) truncates fractional values down to the nearest lower integer. For instance, a duration of 61 minutes yields 61 / 60 = 1 hour, leading to uncollected revenue.

To enforce correct hourly billing rounding rules:

- **Ceiling Division:** int hours = (minutes + 59) / 60;
- **Floating-Point Ceiling:** int hours = (int) Math.ceil((double) minutes / 60.0);

**TASK 3: Design the Control Logic & Pseudocode**

**Execution Strategy**

1. **Allocation Strategy:** Scan multi-floor slots sequentially starting

from Floor 0, Slot 0. Allocate the first available null spot, write the vehicle's registration number, and return [floor, slot]. Return null if the facility is completely full.

2. **Release Routine:** Verify bounds for the requested index [floor, slot]. If occupied, assign the index to null and return true; otherwise return false.

**Fully Executable Source Code (ParkingFacility.java)**

```
package parking.service;

import java.util.Arrays;

public class ParkingFacility {

 private final String[][] slots;

 public ParkingFacility(int[] slotsPerFloor) {

slots = new String[slotsPerFloor.length][];  for

(int f = 0; f < slotsPerFloor.length; f++) {  slots[f]

= new String[slotsPerFloor[f]];  }

 }

 public int[] allocate(String vehicleNo) {
```

```
for (int f = 0; f < slots.length; f++) {

 for (int s = 0; s < slots[f].length; s++) {  if

(slots[f][s] == null) {

 slots[f][s] = vehicleNo;

 return new int[]{f, s};

 }

 }

 }

 return null; // Facility full

 }

 public boolean release(int floor, int slot) {

 if (floor >= 0 && floor < slots.length && slot >= 0 && slot <
slots[floor].length) {

 if (slots[floor][slot] != null) {
```

```
 slots[floor][slot] = null;

 return true;

 }
 }

 return false; // Invalid index or slot already empty  }

 public static void main(String[] args) {

 ParkingFacility p = new ParkingFacility(new int[]{2, 1});

 System.out.println("Allocating TN01AB1111: " +
Arrays.toString(p.allocate("TN01AB1111")));

 System.out.println("Allocating TN02CD2222: " +
Arrays.toString(p.allocate("TN02CD2222")));

 System.out.println("Allocating TN03EF3333: " +
Arrays.toString(p.allocate("TN03EF3333")));

 System.out.println("Allocating TN04GH4444 (Full): " +
Arrays.toString(p.allocate("TN04GH4444")));
```

```
 System.out.println("Release (0,1): " + p.release(0, 1));
 System.out.println("Release (0,1) Again: " + p.release(0, 1));

 System.out.println("Release Out-of-bounds (5,0): " + p.release(5,  0));



 System.out.println("Re-allocating TN04GH4444: " +
Arrays.toString(p.allocate("TN04GH4444")));

 }
```

**Execution Output & Verification (Task 3)**

The following execution output verifies the functionality of the ParkingFacility class implementation:

- allocate("TN01AB1111") -> [0, 0]
- allocate("TN02CD2222") -> [0, 1]
- allocate("TN03EF3333") -> [1, 0]
- allocate("TN04GH4444") -> null (Facility full)
- release(0, 1) + " " + release(0, 1) + " " + release(5, 0) -> true false false
- allocate("TN04GH4444") -> [0, 1]

}

**TASK 4: Design Classes, Objects and Constructors**

**Constructor Chaining via this(...)**

Constructor chaining allows overloaded constructors to delegate initialization logic to a centralized constructor, avoiding duplicated field assignments and ensuring uniform default parameters.public class ParkingTicket {

```
 private final String ticketId;

 private final String regNo;

 private final long entryTimestamp;

 // Primary Constructor

 public ParkingTicket(String ticketId, String regNo, long
entryTimestamp) {

 this.ticketId = ticketId;

 this.regNo = regNo;

 this.entryTimestamp = entryTimestamp;

 }

 // Overloaded Constructor - Defaults timestamp to current system time
 public ParkingTicket(String ticketId, String regNo) {
```

```
this(ticketId, regNo, System.currentTimeMillis());  }

}
```

**Memory Mechanics: Stack vs. Heap Allocation**

- **Stack Memory:** Stores short-lived primitive variable values and object reference pointers during method execution. Stack frames are automatically garbage-collected upon method invocation return.
- **Heap Memory:** Stores actual object instances (ParkingFacility, ParkingTicket, arrays). Objects allocated in the heap remain accessible as long as valid references point to them from stack frames or other active heap objects.

**TASK 5: Apply static Members and the Object Life Cycle**

**Shared State via static Modifiers**

The static keyword associates variables with the overall class definition rather than individual instances. This mechanism provides a central ticket ID sequence counter across the entire system.a

```
public class ParkingTicket {
private static int sequenceCounter = 1000; // Shared state across instances
private final String ticketNumber;public ParkingTicket() {
```

```
 this.ticketNumber = "TKT-" + (++sequenceCounter) ;
}
}
```

### Object Life Cycle and Garbage Collection

1. **Instantiation:** Memory is allocated on the Heap when executing `new ParkingTicket()`.

2. **Active State:** References stored in local stack frames or collection containers prevent removal.

3. **Unreachable State:** When a vehicle exits and its array slot or ticket reference is assigned to `null`, the associated `ParkingTicket` object becomes unreachable.

4. **Garbage Collection (GC):** The JVM Garbage Collector identifies unreferenced objects during background cycles and reclaims their memory allocation automatically.

---

## TASK 6: Design the Inheritance Hierarchy
### Class Diagram Relationship Structure

The class hierarchy uses inheritance to model generic and specialized

vehicle types:

* **`Vehicle`** *(Abstract Base Class)*

 * Extends: `Object`

 * Represents baseline identity (`regNo`).

 * Defines generic abstract methods (`ratePerHour()`).

* **`TwoWheeler`** *(Concrete Class)*

 * Extends: `Vehicle`

 * Encapsulates two-wheeler parking parameters.

 * **`Car`** *(Concrete Class)*

 * Extends: `Vehicle`

 * Encapsulates standard auto properties.

 * **`ElectricCar`** *(Specialized Concrete Class)*
 * Extends: `Car`

 * Implements: `Chargeable`

* Adds EV battery charging routines.

---

## TASK 7: Override and Specialise Behaviour

### Behavioural Customization via Overriding

Specialized subclasses adjust standard rate calculation logic by overriding abstract or base methods. Beyond business domain calculation, overriding core root methods (`toString()`, `equals()`, and `hashCode()`) ensures robust object comparison and hashing within Java collections.

```java

@Override

public boolean equals(Object obj) {

 if (this == obj) return true;

 if (obj == null || getClass() != obj.getClass()) return false;
 Vehicle vehicle = (Vehicle) obj;

 return regNo.equalsIgnoreCase(vehicle.regNo);
```

```
}

@Override

public int hashCode() {

 return regNo.toUpperCase().hashCode();

}
```

**TASK 8: Apply Polymorphism**

**Interface Integration and Dynamic Method Dispatch** Polymorphism allows processing generic Vehicle arrays uniformly while executing specific subclass rate calculations dynamically at runtime.

**Fully Executable Source Code (FeeDemo.java)**

```
package parking.app;

interface Chargeable {

 double chargingFee(int kWh);
}
```

```
abstract class Vehicle {
 private final String regNo;
 protected Vehicle(String regNo) {
 this.regNo = regNo;
 }
 public String getRegNo() {
 return regNo;
 }
 protected abstract double ratePerHour();
 public double calculateFee(int minutes) {
 int hours = (minutes + 59) / 60; // Standard ceiling division
return hours * ratePerHour();
 }
 @Override
```

```
 public boolean equals(Object o) {

 return o instanceof Vehicle && regNo.equalsIgnoreCase(((Vehicle)
o).regNo);

 }

 @Override

 public int hashCode() {

 return regNo.toUpperCase().hashCode();

 }

}

class TwoWheeler extends Vehicle {

 TwoWheeler(String regNo) { super(regNo); }

 @Override protected double ratePerHour() { return 10.0; } }

class Car extends Vehicle {

 Car(String regNo) { super(regNo); }
```

```
 @Override protected double ratePerHour() { return 30.0; } }

class ElectricCar extends Car implements Chargeable {

ElectricCar(String regNo) { super(regNo); }

 @Override public double chargingFee(int kWh) { return kWh * 12.0;  }

}

public class FeeDemo {

 public static void main(String[] args) {

 Vehicle[] parkedVehicles = {

 new TwoWheeler("TN01A1"),

 new Car("TN02B2"),

 new ElectricCar("TN03C3")
 };

 System.out.println("--- Processing Parking Billing ---");  for
```

```
(Vehicle v : parkedVehicles) {

 double totalFee = v.calculateFee(130); // 130 mins -> 3 hours  if (v

instanceof Chargeable c) { // Pattern matching instanceof  totalFee +=

c.chargingFee(10);

 }

 System.out.println(v.getClass().getSimpleName() + " [" +
v.getRegNo() + "] Total: Rs " + totalFee);

 }

 System.out.println("\n--- Identity Equivalence Verification ---");

Vehicle c1 = new Car("TN02B2");

 Vehicle c2 = new ElectricCar("TN02B2");

 System.out.println("Car vs ElectricCar Same Registration Equals? "  +
c1.equals(c2));
 }

}
```

**Execution Output & Verification (Task 8)**

The following execution output verifies the functionality of the FeeDemo class and polymorphic rate calculations:

- TwoWheeler TN01A1 -> Rs 30.0 (130 mins = 3 hours @ Rs 10/hr)
- Car TN02B2 -> Rs 90.0 (130 mins = 3 hours @ Rs 30/hr)
- ElectricCar TN03C3 -> Rs 210.0 (130 mins = 3 hours @ Rs 30/hr + 10 kWh @ Rs 12/kWh)
- Car vs ElectricCar Same Registration Equals? true

**TASK 9: Introduce Abstraction**

**Abstract Class vs. Interface Architectural Boundary** The smart-parking system balances class inheritance and interface implementation based on core system requirements:

- **Abstract Class (Vehicle):** Chosen for state inheritance. It encapsulates common identity properties (regNo) and provides inherited base method implementations (such as duration-to-hours conversion) while forcing subclasses to specify hourly rates.
- **Interface (Chargeable):** Chosen for capability definition without state inheritance. It allows unrelated classes (e.g., ElectricCar, ElectricScooter, or external PowerBank assets) to implement charging capabilities without forcing inheritance from a common root class.

## TASK 10: Enforce Encapsulation through Packages and Access Modifiers

### Package Structure Architecture

The project uses package structures to separate interface logic, business rules, and underlying data models:src/

```
└── parking/

├── model/

│   ├── Vehicle.java

│   ├── TwoWheeler.java

│   ├── Car.java

│   └── ElectricCar.java

├── service/

│   ├── ParkingFacility.java
│   └── TicketManager.java

└── app/
```

└── MainApplication.java

**Modifiers and Defensive Copying**

- **Private Modifiers:** internal array representations (private final String[][] slots) prevent unauthorized direct index modifications from external classes.
- **Defensive Copying:** Methods returning reference configurations return clone arrays or unmodifiable wrapper lists to avoid internal state leakage.

```
public String[][] getFacilityState() {

 String[][] copy = new String[slots.length][];

 for (int i = 0; i < slots.length; i++) {

 copy[i] = slots[i].clone();

 }

 return copy;

}
```

**TASK 11: Implement, Test and Defend the Design**

## TASK 12: Re-evaluate and Adapt the Design

**Open/Closed Principle (OCP) Adaptability Analysis** The Open/Closed Principle states that software components should be open for extension, but closed for modification. The proposed architecture adheres to OCP as follows:

1. **Adding E-Bikes:** Extended by subclassing TwoWheeler and implementing Chargeable without modifying existing Vehicle or ParkingFacility routines.
2. **Peak-Hour Van Surcharges:** Extended by subclassing Car into DeliveryVan and overriding ratePerHour() to compute peak-time surcharges dynamically.
3. **Daily Subscription Passes:** Integrated seamlessly using a strategy design pattern or adding a Pass evaluation step to calculateFee() without altering core storage or allocation mechanics.